```
(1 2)

(a-vals h)
(1 2)
```

All the functions in a.el provide implementation for both associative lists and hash tables. As an example, the *a-keys* function definition is given below:

```
(defun a-keys (coll)
  "Return the keys in the collection
COLL."
  (cond
   ((listp coll)
    (mapcar #'car coll))

   ((hash-table-p coll)
    (hash-table-keys coll))))
```

You can check if two collections are the same using the *a-equal* function. A few examples are as follows:

```
(a-equal collectionA collectionB) ;;
Syntax

(a-equal x (a-list :alpha 1 :bar 2))
nil
(a-equal x (a-list :foo 1 :bar 2))
t

(a-equal h (a-hash-table :a 1 :b 2))
t
```

The *a-has-key?* API takes two arguments – a collection and a key, and returns *true* if the key is present and *nil* otherwise:

```
(a-has-key? collection key) ;; Syntax
(a-has-key? x :foo)
t
(a-has-key? x :gamma)
nil

(a-has-key? h :a)
t

(a-has-key? h :foo)
nil
```

You can count the number of key-value pairs in a collection using the *a-count* function, as shown below:

```
(a-count collection) ;; Syntax

(a-count x)
2

(a-count h)
2
```

The source code of the *a-count* function is as follows:

```
(defun a-count (coll)

  "Count the number of key-value pairs
in COLL.

  Like length, but can also return the
length of hash tables."

  (cond
   ((seqp coll)
    (length coll))

   ((hash-table-p coll)
    (hash-table-count coll))))
```

The *a-assoc* function takes a collection and key-value pairs, and returns an updated collection with the associated values. A couple of examples are shown below:

```
(a-assoc collection key-value-pairs) ;;
Syntax

(a-assoc x :gamma 3)
((:gamma . 3) (:foo . 1) (:bar . 2))

(a-assoc h :c 3)
(:a 1 :b 2 :c 3)
```

The *a-dissoc* function, on the other hand, removes the keys passed as an argument to it and returns a new collection. Examples for the associative list and hash tables are as follows:

```
(a-dissoc collection keys) ;; Syntax

(a-dissoc x :bar)
((:foo . 1))

(a-dissoc h :b)
(:a 1)
```

You can associate new values in a collection for specific keys using the *a-assoc-in* function, as illustrated below:

```
(a-assoc-in collection keys value) ;;

Syntax

(a-assoc-in x [:foo] 10)
((:foo . 10) (:bar . 2))

(a-assoc-in h [:a] 100)
(:a 100 :b 2)
```

The *a-merge* function allows you to merge multiple associative collections. The return type is the type of the first collection. A couple of examples are provided below:

```
(a-merge collections) ;; Syntax

(a-merge x (a-list :gamma 5))
((:gamma . 5) (:foo . 1) (:bar . 2))

(a-merge h (a-hash-table :c 3))
(:a 1 :b 2 :c 3)
```

You can apply a function while merging multiple collections using the *a-merge-with* function. In the following associative list example, the initial value for the key *foo* is *1*, and its value is incremented by *3*. For the hash-table example, the initial value for *b* is *2* and is incremented by *1*.

```
(a-merge-with function collections) ;;
Syntax

(a-merge-with '+ x (a-list :foo 3))
((:foo . 4) (:bar . 2))

(a-merge-with '+ h (a-hash-table :b 1))
(:a 1 :b 3)
```